

Simulating TurtleBot3 AMR on Different Custom Worlds for the Purpose of Data Collection and Record

Prepared by: Belkacem Bekkour
Supervised by: Professor Sousso Kelouwani

1 Running willowgarage.world with TurtleBot3:

For this purpose we have to write our **turtlebot3-willowgarage.launch** custom launch file for the **TurtleBot3 AMR**, which must include the right path for our **.world** file.

Terminal 0: Setup and Directory Navigation

Command 1: Attempt to launch Gazebo ROS

Description: This command tries to access the 'launch' directory within Gazebo ROS. Since 'launch' is a directory, it results in an error.

```
/opt/ros/noetic/share/gazebo_ros/launch
```

Output

```
bash: /opt/ros/noetic/share/gazebo_ros/launch: Is a
directory
```

Command 2: Attempt to access TurtleBot3 Gazebo worlds

Description: This command tries to access the 'worlds' directory for TurtleBot3 in Gazebo, but since it's also a directory, it results in an error.

```
/opt/ros/noetic/share/turtlebot3_gazebo/worlds
```

Output

```
bash: /opt/ros/noetic/share/turtlebot3_gazebo/worlds:
Is a directory
```

Command 3: List available Gazebo worlds

Description: Lists all available Gazebo world files in the specified directory.

```
ls /usr/share/gazebo-11/worlds
```

Output

```
actor_bvh.world
  nested_multilink_shape.world
actor.world
animated_box.world
animation_tension.world
attach_lights.world
  world
...
zephyr_demo.world
openal.world
ortho.world
osrf_elevator.world
pioneer2dx_camera.
```

Command 4: Find 'willowgarage.world' file

Description: Searches for the 'willowgarage.world' file in the '/usr/share' directory.

```
find /usr/share -name "willowgarage.world"
```

Output

```
/usr/share/gazebo-11/worlds/willowgarage.world
```

Terminal 1: Source and Launch Simulation

Command 1: Source the setup file

Description: Sources the setup file to configure the environment for the project.

```
source ~/Desktop/process_LiDAR_data/project_garage/
devel/setup.bash
```

Command 2: Set TurtleBot3 model to 'burger'

Description: Sets the TurtleBot3 model to 'burger' as an environment variable, which is required for launching specific simulations.

```
export TURTLEBOT3_MODEL=burger
```

Command 3: Launch TurtleBot3 in Willow Garage world

Description: Launches TurtleBot3 in a predefined Willow Garage environment using a custom launch file.

```
roslaunch turtlebot3_custom_launch
turtlebot3_willowgarage.launch
```

Output

```
... logging to /home/hedi/.ros/log/4239abf8-a120-11ef
-8a8e-4926e3b2f764/roslaunch-hedi-HP-ProDesk-600-G5
-MT-319569.log
Checking log directory for disk usage. This may take a
while.
Press Ctrl-C to interrupt
Done checking log file disk usage. Usage is <1GB.

started roslaunch server http://hedi-HP-ProDesk-600-G5
-MT:41981/

SUMMARY
=====

PARAMETERS
```

```

* /robot_description: <?xml version="1....
* /rostdistro: noetic
* /rosversion: 1.17.0
* /use_sim_time: True

NODES
/
  gazebo (gazebo_ros/gzserver)
  gazebo_gui (gazebo_ros/gzclient)
  spawn_urdf (gazebo_ros/spawn_model)

auto-starting new master
process[master]: started with pid [319593]
ROS_MASTER_URI=http://localhost:11311
...
[ INFO] [1731434555.399829344, 0.262000000]: DiffDrive
(ns = //): Advertise odom on odom
[spawn_urdf-4] process has finished cleanly
log file: /home/hedi/.ros/log/4239abf8-a120-11ef-8a8e
-4926e3b2f764/spawn_urdf-4*.log

```

Terminal 2: Teleoperation Control

Command 1: Set TurtleBot3 model to 'burger'

Description: Sets the TurtleBot3 model to 'burger', necessary for running specific teleop simulations.

```
export TURTLEBOT3_MODEL=burger
```

Command 2: Launch teleoperation for TurtleBot3

Description: Launches a teleoperation node for controlling TurtleBot3 via keyboard.

```
roslaunch turtlebot3_teleop turtlebot3_teleop_key.
launch
```

Output

```
... logging to /home/hedi/.ros/log/4239abf8-a120-11ef
-8a8e-
Press Ctrl-C to interrupt
Done checking log file disk usage. Usage is <1GB.

started roslaunch server http://hedi-HP-ProDesk-600-G5
-MT:42075/

SUMMARY
=====

PARAMETERS
* /model: burger
* /rostdistro: noetic
* /rosversion: 1.17.0

NODES
/
  turtlebot3_teleop_keyboard (turtlebot3_teleop/
    turtlebot3_teleop_key)

ROS_MASTER_URI=http://localhost:11311

process[turtlebot3_teleop_keyboard-1]: started with
pid [321122]

Control Your TurtleBot3!
-----
Moving around:

      w
    a  s  d
      x

w/x : increase/decrease linear velocity (Burger : ~
      0.22, Waffle and Waffle Pi : ~ 0.26)
a/d : increase/decrease angular velocity (Burger : ~
      2.84, Waffle and Waffle Pi : ~ 1.82)
space key, s : force stop

CTRL-C to quit
```

Terminal 3: Launch Gazebo with RViz

Command 1: Set TurtleBot3 model to 'burger'

Description: Sets the TurtleBot3 model to 'burger', which is used in the Gazebo RViz simulation.

```
export TURTLEBOT3_MODEL=burger
```

Command 2: Launch TurtleBot3 in Gazebo with RViz

Description: Launches TurtleBot3 in a Gazebo simulation environment along with RViz for visualization.

```
roslaunch turtlebot3_gazebo turtlebot3_gazebo_rviz.  
launch
```

Output

```
SUMMARY  
=====
```

```
PARAMETERS  
* /robot_description: <?xml version="1....  
* /robot_state_publisher/publish_frequency: 50.0  
* /robot_state_publisher/tf_prefix:  
* /roscdistro: noetic  
* /rosversion: 1.17.0
```

```
NODES  
/  
  robot_state_publisher (robot_state_publisher/  
    robot_state_publisher)  
  rviz (rviz/rviz)
```

```
ROS_MASTER_URI=http://localhost:11311
```

```
process[robot_state_publisher-1]: started with pid  
[321493]  
process[rviz-2]: started with pid [321494]
```

2 GMapping willowgarage.world with TurtleBot3

Terminal 1: Launch Gazebo with TurtleBot3 in Willow Garage World

Description: In this terminal, we start Gazebo with the Willow Garage world and TurtleBot3.

Command 1: Source the setup file

Description: Sources the setup file to configure the environment for the project.

```
source ~/Desktop/process_LiDAR_data/project_garage/
devel/setup.bash
```

Command 2: Set TurtleBot3 model to 'burger'

Description: Sets the TurtleBot3 model to 'burger' as an environment variable, which is required for launching specific simulations.

```
export TURTLEBOT3_MODEL=burger
```

Command 3: Launch TurtleBot3 in Willow Garage world

Description: Launches TurtleBot3 in a predefined Willow Garage environment using a custom launch file.

```
roslaunch turtlebot3_custom_launch
turtlebot3_willowgarage.launch
```

Terminal 2: Run GMapping for SLAM

Description: This terminal runs the GMapping SLAM node to create a map as the TurtleBot3 navigates through the Willow Garage world.

Command 1: Source the setup file

Description: Sources the setup file to configure the environment for the project.

```
source ~/Desktop/process_LiDAR_data/project_garage/
devel/setup.bash
```

Command 2: Set TurtleBot3 model to 'burger'

Description: Sets the TurtleBot3 model to 'burger'.

```
export TURTLEBOT3_MODEL=burger
```

Command 3: Start the GMapping SLAM node

Description: This command launches the GMapping SLAM node to process lidar data and build the occupancy grid map.

```
roslaunch turtlebot3_slam turtlebot3_slam.launch  
  slam_methods:=gmapping
```

Output

```
... logging to /home/hedi/.ros/log/6239dbb2-a130-11ef  
-8a8e-3936d2e7f764/roslaunch-hedi-HP-ProDesk-600-G5  
-MT-329123.log  
Checking log directory for disk usage. This may take a  
while.  
Press Ctrl-C to interrupt  
Done checking log file disk usage. Usage is <1GB.  
started roslaunch server http://hedi-HP-ProDesk-600-G5  
-MT:42981/  
SUMMARY  
=====  
  
PARAMETERS  
* /slam_methods: gmapping  
* /use_sim_time: True  
* /rostdistro: noetic  
* /rosversion: 1.17.0  
  
NODES  
/  
  slam_gmapping (turtlebot3_slam/turtlebot3_slam)  
  
ROS_MASTER_URI=http://localhost:11311  
  
process[slam_gmapping-1]: started with pid [329210]
```


Terminal 3: Teleoperation Control

Description: This terminal is used to control the TurtleBot3 manually via keyboard teleoperation.

Command 1: Source the setup file

Description: Sources the setup file to configure the environment for the project.

```
source ~/Desktop/process_LiDAR_data/project_garage/  
devel/setup.bash
```

Command 2: Set TurtleBot3 model to 'burger'

Description: Sets the TurtleBot3 model to 'burger', necessary for running teleoperation.

```
export TURTLEBOT3_MODEL=burger
```

Command 3: Launch teleoperation for TurtleBot3

Description: Launches a teleoperation node for controlling TurtleBot3 via keyboard.

```
roslaunch turtlebot3_teleop turtlebot3_teleop_key.  
launch
```

Terminal 4: Visualize Mapping in RViz (Optional)

Description: This terminal is used to visualize the mapping process in RViz.

Command 1: Source the setup file

Description: Sources the setup file to configure the environment for the project.

```
source ~/Desktop/process_LiDAR_data/project_garage/  
devel/setup.bash
```

Command 2: Set TurtleBot3 model to 'burger'

Description: Sets the TurtleBot3 model to 'burger', which is used in RViz for visualization.

```
export TURTLEBOT3_MODEL=burger
```

Command 3: Launch RViz to visualize the map

Description: Launches RViz to visualize the SLAM map as TurtleBot3 moves.

```
roslaunch turtlebot3_slam turtlebot3_slam_rviz.launch
```

Output

```
started roslaunch server http://hedi-HP-ProDesk-600-G5
-MT:44875/

SUMMARY
=====

PARAMETERS
* /robot_description: <?xml version="1....
* /roscdistro: noetic
* /rosversion: 1.17.0

NODES
/
  rviz (rviz/rviz)

ROS_MASTER_URI=http://localhost:11311

process[rviz-1]: started with pid [331213]
```

Terminal 5: Save the Occupancy Grid Map

Description: This terminal is used to save the generated SLAM map.

Command 1: Source the setup file

Description: Sources the setup file to configure the environment for the project.

```
source ~/Desktop/process_LiDAR_data/project_garage/
devel/setup.bash
```

Command 2: Save the map

Description: This command saves the generated map files ('willowgarage_map.pgm' and 'willowgarage_map.yaml') to the Desktop.

```
roslaunch map_server map_saver -f ~/Desktop/
willowgarage_map
```

Output

```
[ INFO] [1731435550.123]: Saving map to ~/Desktop/
willowgarage_map
[ INFO] [1731435551.456]: Map saved as
willowgarage_map.pgm and willowgarage_map.yaml
```

2.1 NOTE:

- If you need to adjust the lidar range, modify the **URDF file**. Look for the tag `<gazebo reference="base_scan">` and change `<max>3.5</max>` under the `<range>` tag to, for example, `<max>15.5</max>`.

```
cd /opt/ros/noetic/share/turtlebot3_description/urdf
```

- To set the range for the occupancy grid in the GMapping Algorithm, you'll need to create a custom **GMapping .launch** that includes the following:

```
sudo nano turtlebot3_burger.gazebo.xacro
```

```
maxRange: 15.5           # Match this to the lidar
  range
minimumScore: 200
delta: 0.05
maxUrange: 15.5          # Maximum usable range for
  mapping
map_update_interval: 2.0 # Update map every 2 seconds
```

- To create a modified launch file that uses custom parameters for GMapping, follow these steps. This new launch file will set each required parameter explicitly and launch TurtleBot3 in your custom environment.

- **Navigate to Your Workspace:** Go to the launch directory in your workspace or create one if it doesn't exist:

```
cd ~/Desktop/process_LiDAR_data/project_garage/src/
  turtlebot3_custom_launch/launch
```

- **Create the New Launch File:** Open a new under the following path `process_LiDAR_data/project_garage/src/turtlebot3_custom_launch` file named `turtlebot3_gmapping_custom.launch`:

```
nano turtlebot3_gmapping_custom.launch
```

- **Add the Following Launch Code:** Paste the following XML code into `turtlebot3_gmapping_custom.launch`:

```
<launch>
  <!-- Arguments -->
  <arg name="model" default="burger" doc="
    model_type_[burger, waffle, waffle_pi]"/>

  <!-- TurtleBot3 Bringup -->
  <include file="$(find turtlebot3_bringup)/
    launch/turtlebot3_remote.launch">
    <arg name="model" value="$(arg model)" />
  </include>

  <!-- GMapping Node with Custom Parameters -->
  <node pkg="gmapping" type="slam_gmapping"
    name="slam_gmapping" output="screen">
    <param name="scan" value="/scan"/>
    <param name="maxRange" value="15.5"/>
    <param name="maxUrange" value="15.5"/>
    <param name="delta" value="0.05"/>
    <param name="linearUpdate" value="0.1"/>
    <param name="angularUpdate" value="0.05"/>
    <param name="map_update_interval" value="
      2.0"/>
    <param name="lskip" value="1"/>
  </node>

  <!-- RViz (optional) -->
```

```

<node pkg="rviz" type="rviz" name="rviz"
      args="-d$(find_turtlebot3_slam)/rviz/
            turtlebot3_gmapping.rviz" />
</launch>

```

- This file launches:
 - * **TurtleBot3 bringup:** Starts up the basic TurtleBot3 nodes.
 - * **GMapping:** Configured with custom parameters for maximum range, map resolution, and update settings.
 - * **RViz:** Automatically opens RViz with the GMapping configuration to visualize the map.
- **Save and Close the File:** Press **Ctrl + X**, then **Y**, and **Enter** to save and exit.
- **Running the New Launch File:** Follow these steps to launch TurtleBot3 with GMapping using the new parameters.

* **Terminal 1: Launch Gazebo with Willow Garage World**

```
source ~/Desktop/process_LiDAR_data/project_garage/
devel/setup.bash
```

```
export TURTLEBOT3_MODEL=burger
```

```
roslaunch turtlebot3_custom_launch
          turtlebot3_willowgarage.launch
```

This terminal launches the Willow Garage world with TurtleBot3 in Gazebo.

* **Terminal 2: Launch GMapping with Custom Parameters**

```
source ~/Desktop/process_LiDAR_data/project_garage/
devel/setup.bash
```

```
export TURTLEBOT3_MODEL=burger
```

```
roslaunch turtlebot3_custom_launch
          turtlebot3_gmapping_custom.launch
```

This terminal launches GMapping with your specified parameters, as well as RViz for visualizing the map.

* **Terminal 3: Teleoperation Control (Optional)** — Use this terminal to control the TurtleBot3 manually.

```
source ~/Desktop/process_LiDAR_data/project_garage/  
devel/setup.bash
```

```
export TURTLEBOT3_MODEL=burger
```

```
roslaunch turtlebot3_teleop turtlebot3_teleop_key.  
launch
```

- **Terminal 4: Save the map** This will generate a **.pgm** file followed by the **.yaml** configuration file for the full occupancy grid map

```
source ~/Desktop/process_LiDAR_data/project_garage/  
devel/setup.bash
```

```
roslaunch map_server map_saver -f ~/Desktop/  
willowgarage_map
```

- **Terminal 5: Run to save the bag file** This will generate a **.bag** file followed by the **topics** configuration file for the full occupancy grid map

```
source ~/Desktop/process_LiDAR_data/project_garage/  
devel/setup.bash
```

```
roslaunch bag_record -O ~/Desktop/process_LiDAR_data/  
project_garage/turtlebot3_slam_data.bag /scan /tf /  
odom /map
```

3 How to Generate a .PGM and .YAML Submap from a .BAG File

This section provides a step-by-step guide to extracting a **.pgm** and **.yaml** submap from a recorded **.bag** file using a custom script.

Steps to Generate the Submap

Step 1: Make the Script Executable

Before running the script, you need to make it executable using the following command:

```
chmod +x extract_maps.sh
```

Step 2: Run the Script

Execute the script in a terminal to begin the process:

```
./extract_maps.sh
```

Step 3: Interactive Prompts

The script will prompt you to input the following:

- **Path to the .bag file:** Enter the full path to the .bag file containing the SLAM data.

```
Enter the path to the BAG file (e.g., /home/hedi/
Desktop/process_LiDAR_data/project_garage/
turtlebot3_slam_data_submap_2.bag):
```

- **Output directory:** Specify the directory where the generated map files will be saved.

```
Enter the output directory (e.g., /home/hedi/Desktop/
process_LiDAR_data/project_garage):
```

Step 4: Verify the Output

Once the script finishes, the map files (.pgm and .yaml) will be saved in the specified output directory. The names of the files will be derived from the .bag file name. For example:

- turtlebot3_slam_data_submap_2.pgm
- turtlebot3_slam_data_submap_2.yaml

Example Run

Below is an example session illustrating the script's execution and output:

```
$ ./extract_maps.sh
Enter the full path to the BAG file (e.g., /home/hedi/
Desktop/process_LiDAR_data/project_garage/
turtlebot3_slam_data_submap_2.bag): /home/hedi/
Desktop/process_LiDAR_data/project_garage/
turtlebot3_slam_data_submap_2.bag
Enter the full path to the output directory (e.g., /
home/hedi/Desktop/process_LiDAR_data/project_garage
): /home/hedi/Desktop/process_LiDAR_data/
project_garage

Launching SLAM node...
Playing bag file: /home/hedi/Desktop/
process_LiDAR_data/project_garage/
turtlebot3_slam_data_submap_2.bag...
Bag file playback completed.
Saving map for turtlebot3_slam_data_submap_2...
Shutting down SLAM node...
Map for turtlebot3_slam_data_submap_2 saved
successfully in /home/hedi/Desktop/
process_LiDAR_data/project_garage.
```

Script Functionalities

This script dynamically handles input and output paths, ensuring flexibility for different `.bag` files and directories. Below are the key functionalities:

- **Launch SLAM Node:** The script initializes a SLAM node to process the data.
- **Play .bag File:** It plays back the recorded `.bag` file to generate mapping data.
- **Save Map:** Upon completing the playback, the script saves the generated map as `.pgm` and `.yaml` files in the specified directory.
- **Clean Exit:** The SLAM node is gracefully shut down after the process.

Usage Notes

- Ensure that the `.bag` file contains the necessary topics for SLAM, such as `/scan`, `/tf`, and `/odom`.
- Verify that the output directory has the required write permissions.

- If the map is not generated as expected, check the `.bag` file for completeness and consistency of data.

4 How to Modify an Existing .PGM Map

This section describes how to modify an existing `.pgm` map using a Python script. This can be useful for adding obstacles, occlusions, or other features to a map for simulation or testing purposes.

Steps to Modify the Map

Step 1: Run the Script

Run the Python script in a terminal to start the map modification process.

```
python3 modify_map.py
```

Step 2: Input File Paths

The script will prompt you for the input file path and the output directory.

- Enter the path to the `.pgm` file you want to modify.

```
Enter the path to the .pgm file you want to modify: /  
home/user/maps/map.pgm
```

- Enter the directory where you want to save the results.

```
Enter the directory where you want to save the results  
: /home/user/maps/modified/
```

Step 3: Modify the Map

After loading the map, a new OpenCV window named `Map Editor` will open.

- **Select a Tool:**
 - Press `t` to select the Rectangle Tool.
 - Press `c` to select the Circle Tool.
 - Press `o` to select the Occlusion Tool.
- **Draw Shapes:**

- Left-click and drag to draw rectangles or circles.
- For occlusions, click on the desired location to add randomized points.

Step 4: Save or Reset

- **Save the Map:** Press **s**.
 - The modified map will be saved in the specified directory with names like `map_modified_1.pgm` and `map_modified_1.yaml`.
- **Reset the Map:** Press **r**.
 - This restores the map to its original state.
- **Quit the Tool:** Press **q**.

Example Usage Report: Modifying a Map

Objective: To add obstacles and occlusions to an existing `.pgm` map for simulation purposes.

Steps Performed:

- **Preparation:**
 - `.pgm` map located at `/home/user/maps/map.pgm`.
 - Output directory: `/home/user/maps/modified/`.
- **Run the Script:**

```
python3 modify_map.py
```

 - Entered `/home/user/maps/map.pgm` for the map file.
 - Entered `/home/user/maps/modified/` for the output directory.
- **Interactive Editing:**
 - Selected Rectangle Tool by pressing **t** and drew a barrier.
 - Switched to Circle Tool by pressing **c** and added circular obstacles.
 - Used Occlusion Tool by pressing **o** to add randomized occlusions.
- **Save the Map:**
 - Pressed **s** to save the map.
 - Output files:
 - * `/home/user/maps/modified/map_modified_1.pgm`
 - * `/home/user/maps/modified/map_modified_1.yaml`
- **Reset (Optional):** Pressed **r** to reset the map.
- **Quit:** Pressed **q** to close the application.

Functionalities in Detail

Key	Functionality Description
t	Rectangle Tool: Draw rectangles (e.g., walls or barriers).
c	Circle Tool: Add circular obstacles.
o	Occlusion Tool: Add randomized points to simulate occlusions.
r	Reset Map: Restore the map to its original state.
s	Save Map: Save the modified map as <code>.pgm</code> and <code>.yaml</code> .
q	Quit: Exit the editor without saving changes.
z	Rotate Map Clockwise: Rotate <code>.pgm</code> Map Clockwise
x	Rotate Map Counter-Clockwise: Rotate <code>.pgm</code> Map Counter-Clockwise

Visualizing the Modified Map in RViz

Follow these steps to visualize the modified `.pgm` map using RViz.

Step 1: Start ROS Core

Initialize the ROS master.

```
roscore
```

Step 2: Load the Modified Map

Use the `map_server` node to serve the map. Replace the path with your modified map's `.yaml` file.

```
roslaunch map_server map_server /home/hedi/Desktop/
process_LiDAR_data/project_garage/modified_map.yaml
```

Step 3: Launch RViz

Start RViz in a new terminal.

```
roslaunch rviz rviz
```

Step 4: Add the Map Topic

- In the RViz interface, locate the "Displays" panel on the left.
- Click **Add**.
- In the pop-up window:

- Select "By Topic".
- Expand the `/map` topic.
- Choose "Map" or a similar option.

Tips for Working in RViz

- **Adjust View:** Use the mouse to pan, zoom, or rotate the camera.
- **Verify Map Properties:** Check the resolution and origin in the `.yaml` file if the map appears distorted or misplaced.
- **Debugging:** Ensure the `map_server` node is running and the file paths in the `.yaml` file are correct if the map does not load.

5 Conclusion

This project successfully demonstrated the simulation of the **TurtleBot3 Autonomous Mobile Robot (AMR)** in custom environments using **Gazebo** and **ROS Noetic**. By performing **SLAM (Simultaneous Localization and Mapping)** in the **Willow Garage world**, we generated realistic occupancy grid maps and visualized the robot's navigation and mapping processes.

Key accomplishments include:

- Launching TurtleBot3 in a custom Gazebo world and implementing **GMapping SLAM**.
- Customizing lidar and SLAM parameters to improve mapping quality.
- Generating and saving maps (`.pgm` and `.yaml`) for further use.
- Enabling manual exploration through teleoperation.

This workflow allows the creation of realistic custom submaps, which can be further utilized for training advanced machine learning models like **Siamese Networks for global localization** and **YOLOv8 for local localization**. The ability to extract and modify maps from simulation enhances the potential for more accurate and robust navigation in real-world scenarios.

Additional Resources:

- GitHub Repository: TurtleBot3 AMR Simulation on Custom Gazebo Worlds ROS1
- Video: TurtleBot3 AMR Simulation On Custom Gazebo Worlds ROS1
- Video: Simulating Added Objects to .pgm Map (Occlusions) - Gazebo — ROS 1 — Python Project